

```

import pandas as pd
import numpy as np
from math import log2

# Dataset
data = pd.DataFrame({
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain', 'Su
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Cool', 'Mild', 'Mild', 'Mild',
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'Normal', 'Nor
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'St
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
})

# Entropy function
def entropy(target):
    values, counts = np.unique(target, return_counts=True)
    ent = 0
    for i in range(len(values)):
        p = counts[i] / sum(counts)
        ent -= p * log2(p)
    return ent

# Information Gain
def info_gain(data, feature, target="PlayTennis"):
    total_entropy = entropy(data[target])
    values, counts = np.unique(data[feature], return_counts=True)

    weighted_entropy = 0
    for i in range(len(values)):
        subset = data[data[feature] == values[i]]
        weighted_entropy += (counts[i]/sum(counts)) * entropy(subset[target])

    return total_entropy - weighted_entropy

# ID3 Algorithm
def id3(data, features, target="PlayTennis"):

```

```
# If all values same
if len(np.unique(data[target])) == 1:
    return np.unique(data[target])[0]

# If no features left
if len(features) == 0:
    return data[target].mode()[0]

# Select best feature
gains = [info_gain(data, f, target) for f in features]
best_feature = features[np.argmax(gains)]

tree = {best_feature: {}}

for value in np.unique(data[best_feature]):
    subset = data[data[best_feature] == value]
    subtree = id3(subset, [f for f in features if f != best_feature], target)
    tree[best_feature][value] = subtree

return tree

# Build tree
features = list(data.columns[:-1])
tree = id3(data, features)

print("Decision Tree:")
print(tree)

# Prediction function
def predict(tree, sample):
    for key in tree.keys():
        value = sample[key]
        subtree = tree[key][value]
        if isinstance(subtree, dict):
            return predict(subtree, sample)
        else:
            return subtree
```

```
# New sample
new_sample = {
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong'
}

result = predict(tree, new_sample)
print("\nPrediction for new sample:", result)
```

Decision Tree:

{'Outlook': {'Overcast': 'Yes', 'Rain': {'Wind': {'Strong': 'No', 'Weak': 'Yes'}}}, 'Sunny': {'Humidity': {'High': 'No',

Prediction for new sample: No

